

ESCOLA SENAI DE ITU
CENTRO EDUCACIONAL CE-125
ENSINO MÉDIO INTEGRADO AO TÉCNICO

CLARCK. Adrian
SANTOS. Fernando. C.
GIANOTTO. Paula. S.
SILVA. Pedro. H. P.

UPLOAD DE ARQUIVOS NA API

SALTO
2026

CLARCK. Adrian
SANTOS. Fernando. C.
GIANOTTO. Paula. S.
SILVA. Pedro. H. P.

Trabalho apresentado à disciplina
Backend do Ensino médio e técnico
da Escola SESI/SENAI de Salto,
como avaliação parcial do 1º
semestre/etapa do 2º ano.
Professor(a): Celso

UPLOAD DE ARQUIVOS NA API

SALTO
2026

Sumário

Documentação Manual - upload de arquivos na - API	2
1.INTRODUÇÃO	2
2.UPLOADS DE ARQUIVOS VIA API:	2
3.MULTER:	2
4.CÓDIGO:	3
5.FONTES:	6

Documentação Manual - upload de arquivos na - API

1.INTRODUÇÃO

O upload de arquivos via API é uma funcionalidade muito utilizada no desenvolvimento de sistemas modernos. Ele permite que aplicações envie arquivos digitais de um dispositivo cliente para um servidor através de requisições HTTP. Esse processo é utilizado em diversas situações do dia a dia, como envio de imagens de perfil, documentos PDF, anexos em sistemas corporativos, uploads de vídeos e armazenamento em nuvem.

2.UPLOADS DE ARQUIVOS VIA API:

O uploads de arquivos via API é uma transferência de arquivos digitais de um dispositivo local para um servidor remoto utilizando uma Interface de Programação de Aplicações (API), utilizando funcionalidades de upload para permitir que usuários enviem documentos, como imagens e PDFs. Ele permite automatizar o envio de documentos de forma rápida, segura e escalável, automatizando o processo, reduzindo erros manuais, interagindo entre sistemas e garantindo um suporte a múltiplos uploads simultaneamente.

Seu funcionamento envolve a comunicação entre o cliente e o servidor. Quando o usuário seleciona um arquivo para upload, a aplicação cliente envia uma requisição HTTP, que contém arquivos e metadados, como o nome do arquivo e o tipo de conteúdo. o servidor processa a requisição, armazena o arquivo em um certo local e retorna a resposta ao cliente.

Neste tipo de sistema o JSON não é utilizado, pois os arquivos não são textos simples, então utilizamos o "multipart/form-data" que permite campos de texto, arquivos e um grande suporte em uma única requisição.

3.MULTER:

O Multer é um Middleware, ou seja, uma “cola” entre diferentes sistemas, utilizado para tratar o "multipart/form-data", auxiliando a receber arquivos, salvar no disco, colocar na memória, limitar o tamanho, filtrar o tipo de arquivo e organizar um upload único ou múltiplo, desse modo, resolvendo a limitação nativa do framework em lidar com uploads de arquivos.

O grupo escolheu essa solução devido ser uma forma mais fácil de compreender e que garante vários benefícios, dentre eles podemos citar um processamento eficiente de dados, uma organização automática da requisição e maior segurança.

4.CÓDIGO:

Para adicionar o multer no nosso sistema, precisamos primeiramente instalar ele com o seguinte comando no terminal: `npm install multer`



Após a instalação, criamos o arquivo multer e o uploads.

[multer.js](#)

```
1  const express = require("express")
2  const multer = require("multer")
3  const path = require("path")
4  const app = express()
5
6  const storage = multer.diskStorage({
7    destination: function(req, file, cb){
8      cb(null, 'uploads/')
9    },
10   filename: function (req, file, cb){
11     cb(null, Date.now() + ".jpg")
12   }
13 })
14
15
16 const upload = multer({
17   storage,
18   limits: {
19     fileSize: 10 * 1024 * 1024,
20   },
21 });
22
23
24 module.exports = upload
```

No arquivo 'multer.js', primeiro foram importados o Express, o Multer e o módulo 'path'. O Express é utilizado para criar a API, o Multer é responsável pelo upload de arquivos e o 'path' ajuda a trabalhar com caminhos de pastas e arquivos.

Depois disso, foi criada a aplicação com 'express()', que inicializa a API. Em seguida, foi criada a constante 'storage' usando 'multer.diskStorage()', responsável por configurar onde e como os arquivos serão salvos.

Na propriedade 'destination', foi definido que os arquivos enviados devem ser armazenados na pasta 'uploads'. Os parâmetros 'req' e 'file' representam a requisição e o arquivo enviado, enquanto 'cb' é uma função callback usada para informar ao Multer o resultado da configuração. O 'null' significa que não houve erro e 'uploads/' indica a pasta de destino.

Na propriedade 'filename', foi definido o nome do arquivo salvo. Foi utilizado 'Date.now()' para gerar um nome único baseado no horário atual, evitando conflito entre arquivos com o mesmo nome. O '+ ".jpg "' adiciona a extensão da imagem.

Depois disso, foi criada a constante 'upload' usando 'multer()', passando a configuração de armazenamento criada anteriormente. Também foi utilizado 'limits' para limitar o tamanho do arquivo em 10 MB, evitando uploads muito grandes.

Por fim, 'module.exports = upload' exporta a configuração para que ela possa ser utilizada nas rotas da API.

[app.js](#)

```

1  const express = require('express')
2  const routes = require('./routes')
3  const path = require("path")
4  const app = express() //cria aplicações web
5  app.use(express.json()) //interpreta requisições json
6  app.use('/', routes)
7  module.exports = app
8
9  app.use('/uploads', express.static(path.join(__dirname, "../..../uploads")))

```

No 'app.js', foi importado o módulo 'path' junto com o Express. A linha 9 foi utilizada para tornar a pasta 'uploads' pública. O 'express.static()' permite acessar os arquivos diretamente pelo navegador. O '__dirname' representa a pasta atual do arquivo e o 'path.join()' monta corretamente o caminho até a pasta 'uploads'. Assim, as imagens enviadas podem ser acessadas por URL através da API.

[ProdutoController.js](#)

```

async cadastrarProduto(req, res){
  try {
    const dadosProduto = {...req.body, imagem : req.file ? req.
    file.filename : null}
    const resultado = await ProdutoService.cadastrarProduto
    (dadosProduto)
    res.json(resultado)
  } catch (erro) {
    res.status (erro.status || 500).json({
      sucesso: false,
      mensagem: erro.mensagem || 'erro interno do servidor',
      erro: erro.stack || erro
    })
  }
}

```

Para adicionar o multer no sistema precisamos modificar o ProdutoController. Neste caso implementamos a linha "`const dadosProduto = {...req.body, imagem : req.file ? req.file.filename : null}`" com ela pegamos a requisição no body, verificamos se há imagem, se houver modificamos o nome, se não deixamos como null.

[ProdutoRouter.js](#)

```
const express = require('express')
const router = express.Router()
const ProdutoController = require('../controllers/ProdutoController')

const upload = require('../config/multer')

router.get('/', ProdutoController.listarProduto)
router.get('/:id', ProdutoController.buscarProdutoPorId)
router.post('/', upload.single('imagem'), ProdutoController.cadastrarProduto)
router.put('/:id', ProdutoController.atualizarProduto)
router.delete('/:id', ProdutoController.deletarProduto)

module.exports = router
```

No Produto Router, onde definimos as rotas, adicionamos a linha “`router.post('/', upload.single('imagem'), ProdutoController.cadastrarProduto)`” a qual recebe a requisição POST e utiliza o Multer para processar o upload da imagem, cadastrando ela no banco de dados.

5.FONTES:

EXPRESS.JS. **Usando o middleware**. Disponível em: <https://expressjs.com/pt-br/guide/using-middleware/>. Acesso em: 1 jun. 2026.

EXPRESSJS. *Multer: Node.js middleware for handling multipart/form-data*. GitHub, 2026. Disponível em: <https://github.com/expressjs/multer>. Acesso em: 1 jun. 2026.

MOZILLA DEVELOPER NETWORK (MDN). *FormData*. Disponível em: <https://developer.mozilla.org/pt-BR/docs/Web/API/FormData>. Acesso em: 1 jun. 2026

STARTDEV. *Como fazer UPLOAD de arquivos com NODE EXPRESS e MULTER*. YouTube. Disponível em: https://youtu.be/_tppRHlrwh8. Acesso em: 1 jun. 2026.